

Secrets Backends for the Nornir Path

FOR DUMMIES

None of this is required — every script in `automation/nornir/` works with zero credential setup, because `containerlab` bakes `ceos`'s default `admin/admin` into the auto-generated inventory. This document is for when you want to practice *not* trusting defaults, and it's also a decent tour of three real Linux secrets-management tools, in increasing order of “what a real team would actually reach for.”

All three work the same way from a script's point of view: they set `NORNIR_USERNAME` / `NORNIR_PASSWORD` in the environment before a script runs, and `creds.py` (imported by every numbered script) overrides the inventory defaults if it finds them set. Only the mechanism that gets them into the environment changes — pick one, or set up all three to compare.

```
./run-with-env.sh    python3 02_nornir_scale.py    # Option A
./run-with-pass.sh  python3 02_nornir_scale.py    # Option B
./run-with-sops.sh  python3 02_nornir_scale.py    # Option C
```

Option A: `.env` file (zero tools)

Plaintext file, gitignored, sourced by a wrapper script (`run-with-env.sh`) that uses `set -a` to auto-export every variable it defines — no library, no encryption, just bash.

```
cp .env.example .env
nano .env          # set NORNIR_USERNAME / NORNIR_PASSWORD
./run-with-env.sh python3 02_nornir_scale.py
```

Good for: a solo lab, or the fastest possible demo of “override the default creds.” **Not good for:** anything that ends up in git — `.env` is gitignored specifically so nobody commits it by accident. There's no encryption here at all; the file is only as safe as your filesystem permissions.

Option B: `pass` — the standard Unix password manager

`pass` stores each secret as a GPG-encrypted file under `~/.password-store`, organized like a filesystem (`pass show network/clab-ceos/username`). It's old-school Unix philosophy: one small tool, does one thing, composes with everything else via `stdout`. If your team already uses GPG keys for anything (signed commits, encrypted email), this reuses that same key.

One-time setup:

```
sudo apt install pass gnupg

# If you don't already have a GPG key:
gpg --full-generate-key
gpg --list-secret-keys --keyid-format long # note the key ID
```

```
pass init <your-gpg-key-id>
```

```
pass insert network/clab-ceos/username # prompts, type: admin
pass insert network/clab-ceos/password # prompts, type: admin
```

Using it:

```
pass show network/clab-ceos/username # prints it, for a sanity check
./run-with-pass.sh python3 02_nornir_scale.py
```

`run-with-pass.sh` just runs `pass show` for both entries and exports the results — edit `PASS_USERNAME_ENTRY` / `PASS_PASSWORD_ENTRY` at the top of that script if you used different paths.

Good for: an individual's personal secrets, or a small team that already shares a GPG web of trust. `pass` has built-in `pass git` support too — your password store can itself be a git repo (encrypted at rest, each secret its own commit history), which is worth showing coworkers as its own small “huh, neat” moment.

Not as good for: teams without existing GPG infrastructure — setting up GPG keys and a trust model from scratch is real overhead, which is partly why `sops` (below) has eaten a lot of `pass`'s use cases in newer, cloud/GitOps-flavored teams.

Option C: `sops` + `age` — encrypt the whole file, diff-friendly

SOPS (originally Mozilla, now a CNCF project) encrypts the *values* in a YAML/JSON/ENV file while leaving the *keys* readable — so `git diff` and code review both still make sense, and the encrypted file is safe to commit. **age** is the modern, simple alternative to GPG it's paired with here: one keypair, no keyring/trust-model ceremony.

Install (Debian ships `age` but not `sops` — no official apt package, grab the `.deb` from GitHub releases):

```
sudo apt install age
```

```
# sops: fetch whatever the current release tag is rather than hardcoding
# a version here (check https://github.com/getsops/sops/releases if curious):
SOPS_VERSION=$(curl -s https://api.github.com/repos/getsops/sops/releases/latest \
  | grep '"tag_name"' | sed 's/.*"v\(.*\)".*/\1/')
curl -LO "https://github.com/getsops/sops/releases/download/v${SOPS_VERSION}/sops_${SOPS_VERSION}_amd64.deb"
sudo dpkg -i "sops_${SOPS_VERSION}_amd64.deb"
```

One-time setup:

```
mkdir -p ~/.config/sops/age
age-keygen -o ~/.config/sops/age/keys.txt
# prints: Public key: age1... <- copy this, it's not secret
```

`keys.txt` is your **private** key — `sops` reads it automatically from that path. It never goes in git. The `age1...` public key is what you encrypt *to*; that one's fine to paste anywhere (README, Slack, a comment in the file itself).

```
cd automation/nornir
cp secrets.example.yaml secrets.enc.yaml
nano secrets.enc.yaml # fill in real values
sops --encrypt --age <your-age1-public-key> -i secrets.enc.yaml
```

`secrets.enc.yaml` is now ciphertext for the values, plaintext for the keys — open it and look, that transparency is the whole selling point over “encrypt the entire file as one opaque blob.”

Using it:

```
./run-with-sops.sh python3 02_nornir_scale.py
```

That uses `sops exec-env`, which decrypts straight into the child process’s environment for the duration of that one command — nothing touches disk unencrypted, nothing lingers in shell history.

Day to day:

```
sops secrets.enc.yaml           # opens decrypted in $EDITOR, re-encrypts on save
sops -d secrets.enc.yaml        # decrypt to stdout, read-only
```

Rotating who can decrypt (adding/removing a coworker’s key) is a config change, not a re-type of every secret:

```
sops --encrypt --age <key1>,<key2>,<key3> -i secrets.enc.yaml # multiple recipients
sops updatekeys secrets.enc.yaml                               # after editing recipients
```

Good for: anything heading toward “more than one person needs access” or “this should live in a shared git repo” — which is most real team secrets. This is the closest of the three to what you’d actually see in a production GitOps/CI pipeline (often backed by AWS/GCP/Azure KMS instead of `age`, but the workflow — encrypted values, readable keys, committed to git — is identical).

Which one should the team actually use?

For this lab specifically: whichever one gets the concept across fastest — `.env` for a 30-second demo, `sops+age` if you want to show something closer to how a real pipeline would do it. For anything beyond the lab — real device credentials, API tokens for production systems — that decision should follow whatever the team already standardizes on for secrets, not get bootstrapped ad hoc per project. If there’s no existing standard, `sops+age` is the more defensible default to propose: it’s diff-friendly, git-native, vendor-neutral, and scales cleanly from “one engineer’s laptop” to “CI pipeline with KMS-backed keys” without changing the workflow, just the backend.